Predicting Chronic Kidney Disease Using a Neural Network Trained on Clinical Indicators

Alexander Bowari
alexander.bowari@icloud.com

Background

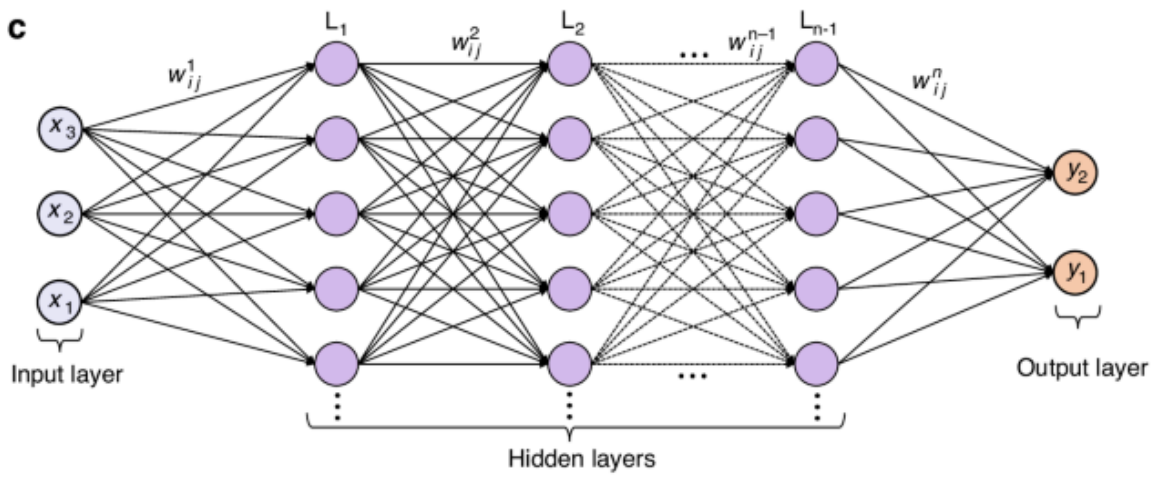
Utilizing resources involving a dataset from the University of California at Irvine, Google Colab, PyTorch, Pandas, SKLearn, and NumPy, a learning algorithm has been successfully curated to detect early signs of chronic kidney disease (commonly medically abbreviated as CDK) with up to 99% accuracy. This algorithm involves a growing development in computer science through the use of a neural network—a machine learning model aiming to mimic the biology of a human brain to process and learn information at a high accuracy. Through later mentioned topics such as forward propagation, backward propagation, and gradient descent, this paper aims to inform you with a fundamental understanding and background about how the math behind the program works to generate code capable of contributing to ongoing medical advances in today's society.

1. Introduction

A neural network is a layered computational model that maps inputs to outputs using weighted connections and nonlinear activation functions. By training on labeled patient data, the model learns to identify patterns in clinical features—such as blood pressure, albumin levels, specific gravity, and hemoglobin—and uses those learned patterns to classify new cases. In the context of chronic kidney

disease prediction, the neural network functions as a pattern recognition system that estimates the probability of disease based on medical indicators.

The structure of this remains fairly simple: the network inputs 24 attributes (which function as neurons) from the user, which branches out to 64 neurons in the first layer, 32 neurons in the second layer, and finally down to 2 classes in the last layer. Like a brain, the goal of training a network like this is to have neurons from one layer correctly affect the right set of neurons onto the next, which continue until the network outputs a prediction. Activation values on the output layer consist of decimal points—the closer it is to one, the higher probability the network thinks that that class distinction is correct. Using this information, the network takes the highest returning value on that activation layer and returns it while also printing the certainty of the network by multiplying that value by 100 to return a percentage.



Tingzhao Fu, Jianfa Zhang, Run Sun, Yuyao Huang, Wei Xu, Sigang Yang, Zhihong Zhu, and Hongwei Chen. 2024. "Optical Neural Networks"

2. Forward Propagation

The process of “guessing” a prediction with the network's parameters is often referred to as forward propagation. In forward propagation, activations from the previous layer influence activation in the next, which continues on until the output layer. These activations are influenced by a weight and bias, which make up 3,746 trainable parameters in the current model.

A weight fundamentally amplifies the activation value of the neuron which gives that value more “weight,” making it a more prominent feature in the next layer of activation—they control the strength

and direction of influence that one neuron has on the next. In the present model, every clinical feature—such as blood pressure, serum creatinine, or hemoglobin—is multiplied by a corresponding weight when passed into the first hidden layer. Weights act as numerical parameters linked between the activations of each neuron, multiplying itself by the activation value in the previous function to contribute to a series of activations in the next layer:

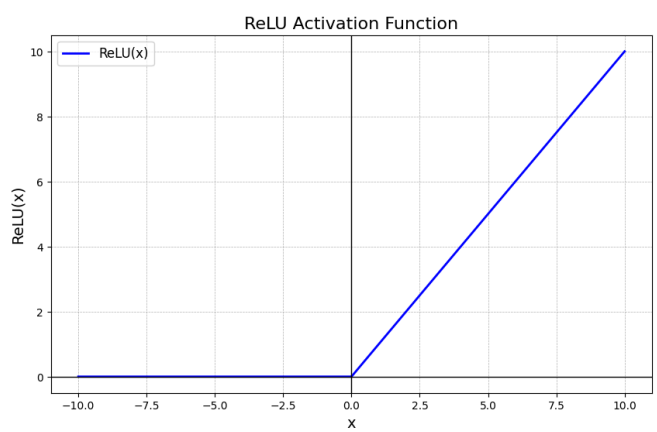
$$a[L] = w^{*}a[L-1]$$

A bias, noted as b, is an additional parameter added to each neuron's weighted sum before an activation function is applied. While weights determine how strongly inputs affect a neuron, biases allow the model to shift the activation threshold which improves flexibility, enabling the network to fit functional patterns that do not pass through the origin. Each neuron in the hidden and output layers contains its own bias term, contributing to the model's overall capacity to represent complex, nonlinear relationships within the dataset:

$$a[L] = w^{*}a[L-1] + b$$

Finally, after each layer we pass the value through an activation function. Nonlinear activation functions are essential components of neural networks because they enable the model to learn complex patterns that cannot be captured through simple linear transformations. Without nonlinearity, a multilayer neural network would force itself mathematically into a single linear transformation regardless of the number of layers. In this case, the model would be incapable of representing the intricate relationships that often exist within clinical data such as diagnosis of CDK.

The two most common activation functions are ReLU and the Sigmoid function. ReLU, or Rectified Linear Unit, introduces nonlinearity by replacing all negative input values to zero while leaving positive values unchanged. This simple mechanism allows the network to model nonlinear relationships while remaining computationally efficient. In the case of this model, ReLU is applied to the hidden layers



GeeksforGeeks. 2025. "ReLU Activation Function in Deep Learning."

of the model to map out those complex interactions between medical features ranging from age to gravity level.

The other aforementioned activation function, the Sigmoid, fundamentally acts as a logistic function that squishes all values between zero and one. More formally put, the values that are largely negative approach a horizontal asymptote of zero, while the more positive values approach a horizontal asymptote of one. The notation for the function, along with the limit statements for the end behavior of the function, goes as follows:

$$\sigma(x) = 1/(1 + e^{-x})$$

$$\lim_{X \to -\infty} \sigma(x) = 0 \qquad \lim_{x \to +\infty} \sigma(x) = 1$$

The function adds the same non-linearity effects that ReLU does to compute more complex patterns, only differing functionally by not making every negative value zero.

Therefore, the total forward propagation process for the "guessing" stage of the network consists first of input features such as blood pressure, serum creatinine, hemoglobin, and other clinical indicators. It then involves an activation from the previous layer, a[L-1], multiplied by the weight connecting that neuron to the next layer, *w*, plus the bias, *b*, that helps determine the strength of an input to a neuron. After those values have been configured, the activation in the current layer is then inputted into an activation function, either Sigmoid or ReLU noted as σ(a[L]), which adds non-linearity to the model. That activation is then passed down to the second layer where the process is repeated again to assign a value to activation a[L+1], which continues through multiple neurons until the output layer. Finally, the output layer—consisting of only two classes in the case of this network for diagnosing CDK—is left with two decimal values. The network will return the higher decimal and note it as the probability or "certainty" of the network's predicted value, successfully guessing through the network's parameters.

3. Backward Propagation

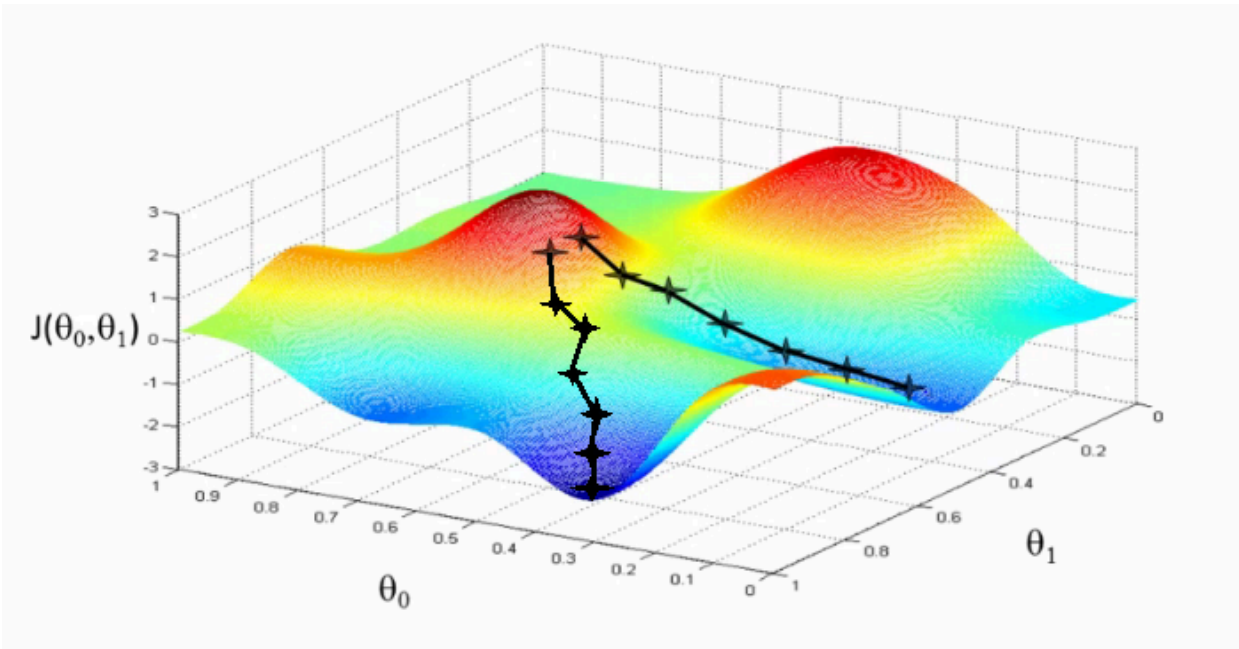
The predictive performance of the chronic kidney disease neural network depends on its ability to learn from labeled clinical data. This learning process is governed by two core mechanisms: backpropagation and gradient descent. Together, these procedures allow the model to iteratively adjust its internal parameters—specifically its weights and biases—to minimize classification error.

After forward propagation, the predicted class of the network is outputted and compared to the true labels using a loss function. In binary classification networks such as this one, the loss function measures the discrepancy between the predicted class probabilities, $\hat{y}$, and the actual diagnosis, y. The loss value represents the magnitude of prediction error for a given batch of patient data. The objective of training is to minimize this loss across the dataset by adjusting the parameters of the neural network, which we do by adjusting the weights and biases.

4. Gradient Descent

The process that many large language models, high performing AI models, and several other machine learning algorithms learn is through gradient descent. Gradient descent is the optimization algorithm responsible for minimizing the loss function; conceptually, it operates by computing the slope, or gradient, of the loss function with respect to each parameter in the network. The gradient indicates both the direction and rate of steepest increase in error. To reduce error, the model updates each parameter in the opposite direction of its gradient. Mathematically, this gradient is computed by taking the derivative of the loss function, also referred to as the cost function, with respect to the weights or biases. The weights or biases are then updated iteratively by subtracting the old weight by the multiplication of the gradient with a learning rate *n*, often consisting of a small decimal. Functionally, this process looks like picking a spot on the loss function and taking small steps down it to reach the bottom.

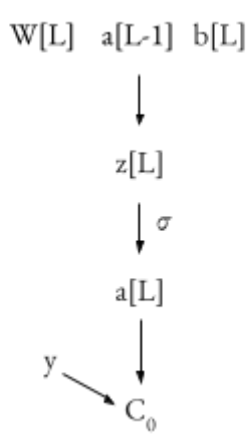
$$W_{new} = W_{old} - n * \partial C/\partial W$$



Riccardo Di Sipio. 2019. "A Quick Guide to Gradient Descent and its Variants."

In the CKD network, this process is applied to all 3,746 trainable parameters. Each parameter is slightly adjusted in order to reduce overall prediction error. Because of the way forward propagation works, a small change in the weights, ∂W, affects a change in the activation function σ[L], which causes a change in a[L], which finally results in a change in the loss function C. Therefore, the use of the chain rule in calculus is necessary to compute the gradient of the loss function with respect to the weights or biases:

$$\partial C/\partial W = (\partial Z/\partial W) * (\partial A/\partial Z) * (\partial C/\partial Z)$$



In computing this derivative with the chain rule, we start with the first function: z[L] = W[L] * a[L-1] + b[L], where z[L] is the value before the activation function. Taking the derivative of this function with respect to its weights simply leaves a[L-1], as this acts as the slope of z[L].

$$\partial Z/\partial W = A[L-1]$$

The second derivative, ∂A/∂Z, comes from taking the derivative of the Sigmoid or ReLu with respect to the initial activation prior to non linearity. Therefore, the partial derivative is just the derivative of the activation function, σ, with the inputted initial function z[L].

$$\partial A/\partial Z = \sigma'(z[L])$$

Finally, we take the partial derivative of the cost function with respect to the activation in the current layer to get ∂C/∂A. Before we take this derivative it is necessary to understand how exactly the model computes the numerical difference or loss of the model's predicted value. This difference is quantified using the mean squared error of the cost function.

For a single prediction, the squared error can be written as:

$$C = (y - \hat{y})^2$$

Here, y represents the true label from the dataset and $\hat{y}$ represents the value predicted by the neural network. The expression computes the difference between the true value and the prediction and then squares that difference. Squaring serves two purposes: it ensures that all errors are positive and it penalizes larger mistakes more heavily than smaller ones.

Intuitively, the cost function acts as a numerical measure of how incorrect the model's prediction is. If the predicted value is close to the true value, $C = (y - \hat{y})^2$ is small and the resulting cost is near zero. Conversely, if the prediction deviates significantly from the true label, the squared error grows rapidly, indicating a larger model error. In this current model, a[L] represents the variable y.

Going back to that function's partial derivative, taking the derivative of the cost with respect a[L] results as follows:

$$\partial C/\partial A = 2(A[L] - y)$$

In total, the overall partial derivative for one training example can be represented as the multiplied partial derivative of all of those function to form the partial derivative of the cost with respect to the weights, returning the direction and size the network must take in minimizing the loss:

$$\partial C/\partial W = a[L-1] * \sigma'(z[L]) * 2(A[L] - y)$$

Accounting for the full cost, we can take the sum of all the partial derivatives of the loss function with respect to the weights and average it out by dividing by the total number of training examples, *x*.

$$\text{Total Cost} = (1/x)\ \Sigma\ [\partial C/\partial W]$$

Performing gradient descent on the biases follows the same logic, except replacing ∂W with ∂b resulting in different overall partial derivatives—the only unique difference is in taking the first initial partial derivative, ∂Z/∂b. Because Z[L] = W[L] * A[L-1] + B[L], taking the derivative of Z[L] will result in the slope of B[L], which happens to be 1. Therefore, the total partial derivative results in:

$$\partial C/\partial b = \sigma'(z[L]) * 2(a[L] - y)$$

As mentioned before, we update the parameters that control our CDK by updating the current weights and biases. This works by taking the old weights, W_{old}, and subtracting them by the learning rate *n* multiplied by the partial derivative of the cost ∂C/∂b. This works the same for the biases as well. The learning rate *n* for this model is significant for determining how big the "step" is in minimizing the loss through the gradient: a big learning rate tends to train quickly however minimizes the ending accuracy of the model, while a small learning rate tends to be more accurate but takes significantly longer to train. In the case of classifying CDK diagnosis, the model is comparatively small with only two hidden layers and 24 input features, and therefore requires a small learning rate without sacrificing significant time in training the actual network.

5. Conclusion

This study presented the development and implementation of a neural network designed to predict the presence of chronic kidney disease using clinical patient data. By leveraging a multilayer feedforward architecture, the model processes twenty-four medical features and learns relationships between physiological indicators and disease outcomes. Through weighted connections, bias adjustments, and nonlinear activation functions, the network is able to represent complex patterns within the dataset that traditional linear models may fail to capture.

The training process relied on backpropagation and gradient descent to iteratively refine the model's parameters. By minimizing the mean squared error between predicted and actual classifications, the network gradually improved its ability to distinguish between patients with and without chronic kidney disease. The use of nonlinear activation functions, particularly the Rectified Linear Unit and

the Sigmoid, enabled the model to learn nonlinear relationships among clinical variables, significantly increasing its expressive power.

Overall, the neural network demonstrates how machine learning techniques can be applied to medical datasets to assist in disease prediction. While such models are not intended to replace clinical diagnosis, they can serve as valuable decision-support tools by identifying patterns that may not be immediately apparent through traditional analysis. Future work may explore larger datasets, more advanced architectures, and improved training strategies to further enhance predictive accuracy and clinical applicability. Lastly, I would like to give a special thanks to the University of California at Irvine for the data on chronic kidney disease features, and a thanks to the teachers at Jesuit Dallas who made my research possible.

Project Link:
https://colab.research.google.com/drive/1W0IHX-UXqQvspN0pYQ3hHnPAwhBLDSQ7?usp=sharing